

CS 213: Data Structures and Algorithms

Midsem Practice Problem Set

~4 hours of practice material • covers Lectures 1–16 (Asymptotics through Heaps)

Instructions

- This set mirrors the style of Quiz 1: short-answer/select-all questions, fill-in-the-code implementation questions, and short proofs, extended to a full midsem's scope and length.
- Point values indicate relative weight/difficulty within this practice set — they are not an official marking scheme.
- Suggested approach: attempt each section un-timed on a first pass, then redo it within the stated time limit closer to the exam.
- Full worked solutions for every question are collected in the Answer Key at the end of this document.

Section overview

- A. Asymptotics & Complexity Analysis — 35 min, 20 pts
- B. Linear Data Structures (Lists, Stacks, Queues, Deques) — 70 min, 30 pts
- C. Trees & Tree Traversal — 55 min, 24 pts
- D. Binary Search Trees — 55 min, 24 pts
- E. Heaps & Priority Queues — 45 min, 20 pts
- F. Mixed Problems & Proofs — 30 min, 16 pts

Total: ~4 hours 50 minutes, 134 points

Section A: Asymptotics & Complexity Analysis

Estimated time: 35 minutes | 20 points

A1. Short Answer & Select-All (10 pts)

A1.1. Which of the following statements are correct? Select all that apply. [2 pts]

- (a) If $f(n) = O(n)$, then $f(n) \leq n$ for all sufficiently large n .
- (b) If $f(n) = O(n)$, then $f(n) = O(n^2)$.
- (c) If $f(n) = \Theta(n)$, then $f(n) = O(n)$ and $f(n) = \Omega(n)$.
- (d) If $f(n) = O(g(n))$ and $g(n) = O(f(n))$, then $f(n) = \Theta(g(n))$.

A1.2. Spot the bug in this definition of Big-O, and fix it. [3 pts]

" $f(n) = O(g(n))$ if there exists a threshold t such that, for every $n > t$, there exists a constant c with $f(n) \leq c \cdot g(n)$."

State precisely what is wrong and give the corrected definition.

A1.3. Rank by growth rate (increasing). [2 pts]

Order these from slowest- to fastest-growing as $n \rightarrow \infty$:

$n \log \log n$, $2^{\log_2 n}$, $n^{1.5}$, $\sqrt{n}$, $n!$, $n \log n$, 3^n , n^3

(Hint: simplify $2^{\log_2 n}$ first.)

A1.4. Given two singly linked lists of n elements each (you hold head and tail pointers for both), what is the tightest worst-case bound to produce a single list containing all elements of both (duplicates allowed)? [3 pts]

- $O(1)$
- $O(\log n)$
- $O(n)$
- $O(n^2)$

Justify your choice in one sentence.

A2. Proofs (6 pts)

A2.1. Prove or disprove: if $f(n) = O(n^2)$, then $f(n) \neq \Omega(n^2)$. [3 pts]

A2.2. Prove: $O(n^2 + 3n - 5) = O(n^2)$. [3 pts]

A3. Analysis & Recurrences (4 pts)

A3.1. State the tightest Big-O bound for the total number of iterations of the inner loop: [2 pts]

```
void mystery(int n) {
    for (int i = 1; i <= n; i *= 3) {
        for (int j = 0; j < i; j++) {
            std::cout << i << " ";
        }
    }
}
```

A3.2. Solve using the Master Theorem (state the case used): [2 pts]

- (a) $T(n) = 4T(n/2) + n$
- (b) $T(n) = 2T(n/2) + n \log n$

Section B: Linear Data Structures — Lists, Stacks, Queues, Deques

Estimated time: 70 minutes | 30 points

B1. Warm-up (8 pts)

B1.1. Trace the stack (LIFO). Show contents after each operation: [2 pts]

```
push(4); push(9); push(1); pop();  
push(6); push(2); pop(); pop(); push(8);
```

B1.2. Trace the queue (FIFO). Show contents after each operation: [2 pts]

```
enqueue(15); enqueue(25); dequeue();  
enqueue(35); enqueue(45); dequeue(); dequeue();
```

B1.3. Using a stack, determine whether each string has balanced brackets. Give a one-line reason for each 'no'. [2 pts]

(a) "{(())}" (b) "{{} }" (c) "(())(" (d) "{[[[(())]]}"

B1.4. Write pseudocode to reverse a singly linked list in $O(n)$ time and $O(1)$ extra space. State the invariant maintained by your loop. [2 pts]

B2. Amortized Analysis (10 pts)

B2.1. A dynamic array starts at capacity 1. Each time it is full, a new array of capacity $\text{old_capacity} + 4$ is allocated and all elements copied over (growth by a constant, not doubling). [4 pts]

- (a) What is the total cost of copying while inserting n elements, in terms of n ?
- (b) What is the amortized cost per insertion? Is `push_back` still $O(1)$ amortized under this scheme?
- (c) For $n = 1,000,000$, roughly how many times more total copying does this scheme do compared to doubling?

B2.2. Each time a `std::vector` runs out of space, a new vector of triple the capacity is allocated and the data is moved over. What is the tightest bound on the total time to insert n elements? [3 pts]

- $O(1)$
- $O(n)$
- $O(n \log n)$
- $O(n^2)$

B2.3. Consider the Deque implementation from lecture: a directory of pointers to fixed-capacity chunks, plus a directory that itself doubles when full. Select all correct statements about this design. [3 pts]

- (a) `push_front` / `push_back` / `pop_front` / `pop_back` all run in worst-case $O(1)$ time.
- (b) The worst case for any single operation occurs when the directory itself must be reallocated, costing $\Theta(\text{current size})$.
- (c) Random access (`operator[]`) takes worst-case $O(1)$ time.
- (d) Only the directory is ever reallocated; the individual chunks holding elements are fixed size and never reallocated.

B3. Implement It (8 pts)

B3.1. Complete the missing `dequeue()` body below so that a queue is built from two stacks, with `enqueue` $O(1)$ worst-case and `dequeue` $O(1)$ amortized. Then argue why the amortized cost is $O(1)$ — one or two sentences using a potential/charging argument. [5 pts]

```
class QueueUsingStacks {  
    std::stack<int> in_stack, out_stack;  
public:
```

```

void enqueue(int x) { in_stack.push(x); }

int dequeue() {
    if (out_stack.empty()) {
        // <-- fill this in -->
    }
    int val = out_stack.top();
    out_stack.pop();
    return val;
}
};

```

B3.2. A deque is to be built from a single `std::stack` (only one stack, no second container, no array). Explain why `push_back` and `push_front` cannot both be supported in $O(1)$ amortized time using just one stack — or give a scheme that does it, with its complexity. [3 pts]

B4. Medium Problems (4 pts)

B4.1. Merge two sorted singly linked lists into one sorted list in-place (reusing existing nodes, not creating new ones). State the time and space complexity. [2 pts]

L1: 2 -> 5 -> 8

L2: 1 -> 3 -> 9 -> 10

Output: 1 -> 2 -> 3 -> 5 -> 8 -> 9 -> 10

B4.2. Given a sorted singly linked list, remove all nodes whose value appears more than once, so only values that were unique in the input remain (not just de-duplication). [2 pts]

Input: 1 -> 2 -> 2 -> 3 -> 4 -> 4 -> 5

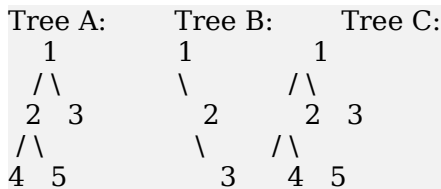
Output: 1 -> 3 -> 5

Section C: Trees & Tree Traversal

Estimated time: 55 minutes | 24 points

C1. Basics (8 pts)

C1.1. For each tree below, give its height (edges from root to deepest leaf), node count, and say whether it is height-balanced. [3 pts]



C1.2. Write the output of each traversal of this tree: [3 pts]



In-order: Pre-order: Post-order: Level-order:

C1.3. For a complete binary tree of height h (root at height h , a single leaf has height 0): give the minimum and maximum possible number of nodes, and, given n nodes, the height in terms of n . [2 pts]

C2. Construction & Manipulation (10 pts)

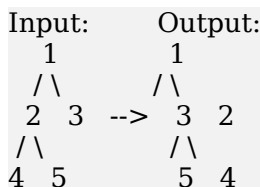
C2.1. Given the pre-order and in-order traversals below, reconstruct the unique binary tree they came from. Show your recursive splitting step, not just the final picture. [4 pts]

Pre-order: G B A D C E H F I

In-order: A B C D E G F H I

C2.2. A binary tree's post-order and in-order traversals are given. Is a binary tree with these two traversals always uniquely determined? Give a proof sketch or a counterexample with two different trees sharing both traversals. [3 pts]

C2.3. Write a function to swap the left and right children of every node in a binary tree (mirror the tree). State the time and space complexity of your solution, including recursion-stack space. [3 pts]



C3. Harder Problems (6 pts)

C3.1. Given a binary tree (not necessarily a BST) and two node references u, v , describe an $O(n)$ -time algorithm to find their lowest common ancestor. Briefly justify the $O(n)$ bound. [3 pts]

C3.2. Design serialize / deserialize functions for a general binary tree (nodes may have 0, 1, or 2 children) such that $\text{deserialize}(\text{serialize}(T))$ always reconstructs T exactly. Explain why pre-order alone is not enough, and what extra information your encoding adds to fix that. [3 pts]

```
std::string serialize(TreeNode* root);  
TreeNode* deserialize(const std::string& data);
```

Section D: Binary Search Trees

Estimated time: 55 minutes | 24 points

D1. Building & Searching (8 pts)

D1.1. Insert the following keys, in this order, into an initially empty BST: 15, 6, 20, 3, 9, 18, 22, 2, 7, 13. Draw the resulting tree and state its height. [3 pts]

D1.2. Using the BST built in D1.1, trace the search path (sequence of keys visited) for: 13, 8, 22, 1. How many comparisons does each search take? [2 pts]

D1.3. Using the BST from D1.1, show the tree after each deletion is performed in sequence: delete 2, then delete 6, then delete 15. Name which of the three deletion cases (leaf / one child / two children) applies each time, and which convention you use when a node has two children. [3 pts]

D2. Core Properties (8 pts)

D2.1. Implement `isBST(root)`, which checks whether a given binary tree satisfies the BST property. Explain with an example why checking only "left child < node < right child" locally, without passing down a valid range, is insufficient. [4 pts]

```
bool isBST(TreeNode* root);
```

D2.2. Given a sorted array of n distinct keys, build a BST of minimum possible height. State the height you achieve and why it is optimal. Then argue what goes wrong (in terms of height) if you instead insert the same n keys one at a time, in sorted order, into an empty BST. [4 pts]

Input: [2, 4, 6, 8, 10, 12, 14]

D3. Harder Problems (8 pts)

D3.1. Given a BST with n nodes and a range $[a, b]$, find all keys in that range. Give pseudocode and analyze the running time in terms of n and k , the number of keys reported — in particular, explain why a well-designed algorithm need not visit every one of the n nodes. [4 pts]

```
    12
   / \
  6   18
 / \ / \
3  9 15 22
```

Query [7, 16] should report: 9, 12, 15

D3.2. You are given a queue containing a pre-order traversal of a BST (all keys distinct), and told the maximum possible key value is bound. Reconstruct the BST in $O(n)$ time (not $O(n \log n)$). Explain precisely why passing a bound parameter down the recursion avoids the need to compare against every ancestor. [4 pts]

```
TreeNode* buildBST(std::queue<int>& preorder, int bound);
```

Example pre-order: 12, 6, 3, 9, 18, 15, 22

Section E: Heaps & Priority Queues

Estimated time: 45 minutes | 20 points

E1. Basics (6 pts)

E1.1. For each array below (0-indexed, children of i at $2i+1$, $2i+2$), say whether it is a valid min-heap. If not, name one violated pair. [2 pts]

(a) [2, 4, 3, 8, 5, 9, 7]

(b) [2, 5, 3, 4, 8, 9, 7]

(c) [2, 4, 3, 1, 5, 9, 7]

E1.2. Starting from the array [11, 4, 8, 2, 9, 1, 6, 3], build a min-heap using the standard bottom-up heapify procedure. Show the array after processing each non-leaf node. [2 pts]

E1.3. For a complete binary min-heap stored in an array, holding 500 elements: what is its height? What is the index range occupied by the leaves? [2 pts]

E2. Applying Heaps (8 pts)

E2.1. Given an unsorted array of n numbers and an integer k , describe the most efficient approach (of the three below) to find the k -th largest element, and give its time complexity: [3 pts]

- Sort the entire array, then index into it
- Maintain a min-heap of size k while scanning the array once
- Use the QuickSelect partitioning algorithm

Explain when QuickSelect's worst case is worse than the heap approach, and why the heap approach is preferable when the array is a stream you can only pass over once.

E2.2. Given k sorted linked lists with a total of n elements across all of them, merge them into one sorted list using a min-heap of size k . Give the algorithm and derive its time complexity in terms of n and k . [3 pts]

L1: 2 -> 6 -> 10

L2: 1 -> 5 -> 9

L3: 3 -> 4 -> 8

E2.3. Given a stream of integers and a fixed window size k , describe how a data structure other than a plain heap (name it) achieves $O(n)$ total time to report the maximum of every window, beating the $O(n \log k)$ a heap-based approach would give. [2 pts]

Input: [4, 2, 6, 1, 8, 3], $k = 3$

Output: [6, 6, 8, 8]

E3. Design Problems (6 pts)

E3.1. Design a MedianFinder that supports `addNum(x)` and `findMedian()` efficiently over a stream, using two heaps (one max-heap, one min-heap). State the invariant you maintain between the two heaps' sizes, and the complexity of each operation. [3 pts]

```
class MedianFinder {
public:
    void addNum(int num);
    double findMedian();
};
```

E3.2. You need a priority queue that supports `insert(key, priority)`, `extract_max()`, and `decrease/increase` an existing key's priority (given a reference/handle to it) — all in $O(\log n)$. A plain binary-heap array does not support the last

operation efficiently without extra bookkeeping. What extra piece of information would you store, and how does it let you locate and fix up an arbitrary element in $O(\log n)$? [3 pts]

Section F: Mixed Problems & Proofs

Estimated time: 30 minutes | 16 points

F1. Choosing Data Structures (6 pts)

F1.1. For each scenario, name the single best-suited data structure from this course and justify it in one sentence: [4 pts]

- (a) Repeatedly need the most urgent of a changing set of tasks.
- (b) Need to support undo and redo of the last k actions in a text editor.
- (c) Need to check, for up to a million IDs, whether a given ID has been seen before, as fast as possible on average.
- (d) Need to keep a running top-10 of the largest values seen so far in a stream of a billion numbers.

F1.2. A colleague claims: "since `std::vector::push_back` is $O(1)$, an algorithm that calls `push_back` inside a loop over n iterations always runs in $O(n)$ total." Is this reasoning sound? Explain, distinguishing worst-case from amortized cost. [2 pts]

F2. Proof (4 pts)

F2.1. Prove that finding both the minimum and the maximum of an array of n elements requires at least $\lceil (3n/2) \rceil - 2$ comparisons in the worst case, and describe an algorithm (processing elements in pairs) that matches this bound. [4 pts]

F3. Harder Analysis (6 pts)

F3.1. Count the number of inversions in an array — pairs (i, j) with $i < j$ but $A[i] > A[j]$. The straightforward approach is $O(n^2)$. Describe a divide-and-conquer approach (based on merge sort) that solves it in $O(n \log n)$, and explain where in the merge step the inversions are counted. [3 pts]

Input: [2, 4, 1, 3, 5]

Output: 3 (pairs: (2,1), (4,1), (4,3))

F3.2. Analyze the total time complexity of the algorithm below, in terms of $n = \text{arr.size()}$, justifying each term: [3 pts]

```
void repeatedRebuild(std::vector<int>& arr) {
    for (int i = 0; i < arr.size(); i++) {
        BST bst;
        for (int j = 0; j <= i; j++)
            bst.insert(arr[j]); // O(height) per insert
        std::cout << bst.kthSmallest(i / 2);
    }
}
```

Consider both the case where insertions keep the BST balanced and the worst case where they don't.

Answer Key

Section A: Asymptotics & Complexity Analysis

A1.1

Correct: (b), (c), (d). Statement (a) is false — e.g. $f(n)=2n$ is $O(n)$ but $f(n) > n$ for all $n > 0$. Big-O is only an upper bound up to a constant factor, so it does not forbid f from exceeding n itself.

A1.2

The bug: the constant c is allowed to depend on n (it is chosen "for every $n > t$ ", i.e. after n is fixed). A single c must work for all $n > t$ simultaneously.

Fixed: $f(n) = O(g(n))$ if there exist a threshold t and a constant c such that, for every $n > t$, $f(n) \leq c \cdot g(n)$. (The order in which t and c are introduced does not matter, as long as c is fixed before n is allowed to range over all values $> t$.)

A1.3

Since $2^{\lceil \log_2 n \rceil} = n$, the order is:

$$\sqrt{n} < n \log \log n < n \log n < 2^{\lceil \log_2 n \rceil} (= n) < n^{1.5} < n^3 < 3^n < n!$$

Common mistake: forgetting to simplify $2^{\lceil \log_2 n \rceil}$ to n first — it belongs with the linear-growth functions, not the exponentials.

A1.4

$O(1)$. Since both head and tail pointers are available for each list, simply set L1's tail->next = L2's head, and the new combined list runs from L1's head to L2's tail. No traversal of either list is needed — just two pointer updates. (The bound would be $O(n)$ if only head pointers were given, since reaching the end of L1 would require a full traversal.)

Section A: Proofs

A2.1

Disprove. Counterexample: $f(n) = n^2$. Clearly $f(n) = O(n^2)$ (take $c = 1$). But also $f(n) = \Omega(n^2)$ (again take $c = 1$), since $n^2 \geq 1 \cdot n^2$ for all n . So $f(n) = O(n^2)$ does not imply $f(n) \neq \Omega(n^2)$ — in fact a function can be, and often is, both at once (that's exactly what $\Theta(n^2)$ means).

A2.2

For $n \geq 1$: $n^2 + 3n - 5 \leq n^2 + 3n^2 = 4n^2$ (since $3n \leq 3n^2$ and $-5 \leq 0$ for $n \geq 1$). So with $c = 4$ and threshold $t = 1$, $n^2 + 3n - 5 \leq c \cdot n^2$ for all $n > t$. Hence $O(n^2 + 3n - 5) = O(n^2)$.

Section A: Analysis & Recurrences

A3.1

Outer loop runs for $i = 1, 3, 9, \dots$, up to $\leq n$: that's $\lceil \log_3 n \rceil + 1$ iterations. Inner loop runs i times. Total inner-loop iterations = $1 + 3 + 9 + \dots + 3^{\lceil \log_3 n \rceil} \leq (3/2) \cdot n$ (geometric series dominated by its last term) = $O(n)$.

A3.2

(a) $T(n) = 4T(n/2) + n$: $a = 4$, $b = 2$, $f(n) = n$. $n^{\log_b a} = n^2$. Since $f(n) = O(n^{2-\epsilon})$, this is Master Theorem Case 1 $\rightarrow T(n) = \Theta(n^2)$.

(b) $T(n) = 2T(n/2) + n \log n$: $a = 2$, $b = 2$, $n^{\log_b a} = n$. $f(n) = n \log n = n^1 \cdot \log n$, which is Case 2's extended form ($f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$ with $k = 1$) $\rightarrow T(n) = \Theta(n \log^2 n)$.

Section B: Linear Data Structures

B1.1

After each op: $[4] \rightarrow [4,9] \rightarrow [4,9,1] \rightarrow [4,9] \rightarrow [4,9,6] \rightarrow [4,9,6,2] \rightarrow [4,9,6] \rightarrow [4,9] \rightarrow [4,9,8]$. Final stack (top last): $[4, 9, 8]$.

B1.2

After each op: [15] → [15,25] → [25] → [25,35] → [25,35,45] → [35,45] → [45]. Final queue (front..back): [45].

B1.3

(a) "[{()})]" — No: after "[{(", the next closing bracket ")" doesn't match the top of stack "(".

(b) "[{()})]" — Yes: every bracket closes in LIFO order correctly.

(c) "(()))(": No: the 4th character closes an extra ")" when the stack has only "(" from position 1 left (net: closes more than opened at that point), and a trailing unmatched "(" remains at the end.

(d) "[[[[(())]]]]": No: the "(" opened is closed by "]", a bracket-type mismatch.

B1.4

Maintain `prev = nullptr`, `curr = head`. Repeatedly: save `next = curr->next`; set `curr->next = prev`; advance `prev = curr`, `curr = next`. Loop invariant: at the start of each iteration, the sublist ending at `prev` (inclusive) is already reversed and detached from the rest of the list rooted at `curr`. When `curr` becomes null, `prev` is the new head. $O(n)$ time, $O(1)$ space.

B2.1

(a) Capacities used: 1, 5, 9, 13, ..., each requiring a copy of all elements present so far when the resize happens. This is an arithmetic sequence of copy-costs, summing to $O(n^2/\text{constant}) = O(n^2)$.

(b) Amortized cost per insertion = $O(n^2)/n = O(n)$ — linear, i.e. NOT $O(1)$ amortized. Growing by a fixed additive constant is asymptotically bad.

(c) Doubling does $O(n)$ total copying (amortized $O(1)/\text{insert}$); constant growth does $O(n^2)$. For $n = 10^6$, that's roughly a factor of $\sim n = 10^6$ times more total copying work — a huge practical difference.

B2.2

$O(n)$. Tripling is a constant multiplicative factor just like doubling; the geometric series $1+3+9+\dots+n$ sums to $O(n)$, giving $O(1)$ amortized per insertion and $O(n)$ total.

B2.3

Correct: (b) and (c). (a) is false — push/pop at the ends are $O(1)$ amortized, not worst-case, precisely because of the case in (b). (d) is false — the individual chunks are also periodically reallocated/replaced as elements are added at either end (only the directory's reallocation pattern was described in the question, but the fixed-capacity chunks themselves get swapped in/out at the two ends, not permanently fixed for the whole run).

B3.1

```
if (out_stack.empty()) {
    while (!in_stack.empty()) {
        out_stack.push(in_stack.top());
        in_stack.pop();
    }
}
```

Argument: each element is pushed onto `in_stack` once ($O(1)$) and, over its lifetime, moved to `out_stack` exactly once and popped from `out_stack` exactly once ($O(1)$ each). So across any sequence of n enqueue/dequeue calls, the total work done transferring elements is $O(n)$, giving $O(1)$ amortized cost per operation (charge each element's $O(1)$ transfer cost to its own enqueue).

B3.2

With a single stack, only one end can be accessed in $O(1)$ — the top. Supporting `push_front` and `push_back` both efficiently needs access to two logical ends; a single LIFO stack fundamentally only exposes one. One can simulate the other end by reversing the whole stack onto a temporary structure, but that costs $O(n)$ per operation on the 'far' end (or $O(n)$ amortized only under restrictive access patterns) — so, in general, one stack alone cannot give $O(1)$ (or even $O(1)$ amortized under arbitrary interleavings) for both ends; a second stack (as in the two-stack deque) is needed to get $O(1)$ amortized on both ends.

B4.1

Standard merge using a dummy head: repeatedly compare the current fronts of L1, L2 and splice the smaller node onto the output, advancing that list's pointer, without allocating new nodes. Time: $O(n + m)$. Space: $O(1)$ extra (pointer manipulation only; recursion would add $O(n+m)$ stack space if done recursively).

B4.2

Use a dummy node before head. Walk with prev and curr; whenever curr's value equals curr->next's value, skip forward past all nodes with that value (unlinking them all, including the first occurrence), and set prev->next to the node after that run; otherwise, advance prev and curr normally. $O(n)$ time, $O(1)$ space.

Section C: Trees & Tree Traversal

C1.1

Tree A: height 2, 5 nodes, balanced (heights of subtrees under root differ by ≤ 1 and each subtree itself balanced).

Tree B: height 2, 3 nodes, but it is a right-leaning... actually a left/degenerate chain (1 -> 2 -> 3) — height 2 with only 3 nodes means it's a skewed path, which is NOT balanced by the usual AVL-style definition (left subtree height 1, right subtree height -1, difference 2).

Tree C: height 1, 3 nodes (root + 2 children), balanced.

C1.2

In-order: D B E A F C Pre-order: A B D E C F Post-order: D E B F C A Level-order: A B C D E F

C1.3

Minimum nodes for height h : $h + 1$ (a single path down). Maximum nodes for height h : $2^{h+1} - 1$ (a full complete tree). Given n nodes, height ranges from $\lceil \log_2(n+1) \rceil - 1$ (complete) up to $n - 1$ (skewed) — since the question asks specifically about a complete binary tree of height h , its node count n satisfies $2^h \leq n \leq 2^{h+1} - 1$, so $h = \lfloor \log_2 n \rfloor$.

C2.1

Pre-order's first element, G, is the root. Locate G in the in-order sequence: A B C D E | G | F H I — everything to its left (A B C D E) forms the left subtree, everything to its right (F H I) forms the right subtree.

Left subtree: pre-order restricted to {A,B,C,D,E} is B A D C E, so its root is B; splitting its in-order (A B C D E) at B gives left={A}, right={C D E}. Within {C,D,E}, pre-order is D C E, so root D, splitting in-order C D E at D gives left={C}, right={E}. So the left subtree is B, with left child A and right child D(left C, right E).

Right subtree: pre-order restricted to {F,H,I} is H F I, so root H; splitting in-order F H I at H gives left={F}, right={I}. So the right subtree is H, with left child F and right child I.



C2.2

No — post-order + in-order together DO uniquely determine the tree (this pair, like pre-order + in-order, is always sufficient), because post-order's last element gives the root, and in-order splits into left/right subtrees exactly as with pre-order, just reading the root from the end instead of the start. What is NOT sufficient is pre-order + post-order alone (without in-order): for example, a single root with only a left child vs. only a right child can produce ambiguity, since without in-order there's no way to tell which side an only-child belongs on from pre-order+post-order alone.

C2.3

```

TreeNode* invert(TreeNode* root) {
    if (!root) return nullptr;
    std::swap(root->left, root->right);
    invert(root->left);
    invert(root->right);
}

```

```

return root;
}

```

Time: $O(n)$ — every node visited once. Space: $O(h)$ recursion-stack space, where h is the tree height ($O(\log n)$ balanced, $O(n)$ worst-case skewed); no extra heap allocation needed since children are swapped in place.

C3.1

Recursively define $\text{find}(\text{node}, u, v)$: if node is null or node is u or v , return node. Recurse into left and right children. If both recursive calls return non-null, node itself is the LCA (u and v are in different subtrees). If only one side returns non-null, propagate that result up. This visits each node at most once, so it is $O(n)$.

C3.2

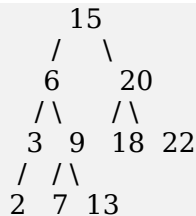
Pre-order alone is not enough for a general binary tree because it doesn't distinguish 'only a left child' from 'only a right child', or tell you where one subtree ends and the next begins without additional markers — unlike a BST, where the key values themselves (via comparisons) tell you the structure.

Fix: encode null children explicitly, e.g. pre-order traversal where every null pointer is written as a sentinel token (e.g. "#"), separated by a delimiter. Deserializing then reads tokens in the same pre-order, recursively building a node and its two children (recurring even into the '#' markers, which simply return null and consume one token), so the structure is fully and unambiguously recoverable in one linear pass.

Section D: Binary Search Trees

D1.1

Insertion order 15, 6, 20, 3, 9, 18, 22, 2, 7, 13 produces:



Height = 3 (edges from root 15 down to e.g. 2, or to 13).

D1.2

Search 13: $15 \rightarrow 6 \rightarrow 9 \rightarrow 7 \rightarrow 13$ (5 comparisons, found).

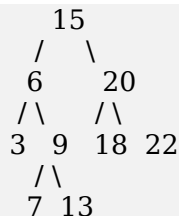
Search 8: $15 \rightarrow 6 \rightarrow 9 \rightarrow 7 \rightarrow$ (7's right is null) not found (5 comparisons).

Search 22: $15 \rightarrow 20 \rightarrow 22$ (3 comparisons, found).

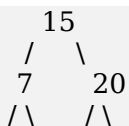
Search 1: $15 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow$ (2's left is null) not found (4 comparisons).

D1.3

Delete 2 (leaf): simply remove it. Node 3 now has no children.



Delete 6 (two children: 3 and 9): in-order successor = minimum of right subtree rooted at 9 = 7. Replace 6's value with 7, then remove the original leaf node 7.



```

  3  9  18  22
   \
   13

```

Delete 15 (root, two children): in-order successor = minimum of right subtree rooted at 20 = 18. Replace 15's value with 18, then remove the original leaf node 18.

```

      18
     /  \
    7    20
   /\   \
  3  9   22
   \
   13

```

D2.1

```

bool isBSTHelper(TreeNode* node, long lo, long hi) {
    if (!node) return true;
    if (node->val <= lo || node->val >= hi) return false;
    return isBSTHelper(node->left, lo, node->val)
        && isBSTHelper(node->right, node->val, hi);
}
bool isBST(TreeNode* root) { return isBSTHelper(root, LONG_MIN, LONG_MAX); }

```

Why local checks fail: a node might satisfy left-child < node < right-child relative to its immediate parent, yet still violate the BST property with a more distant ancestor. Example: root 10 with right child 15, whose left child is 8 — locally, 8 < 15 holds, but 8 is in 10's right subtree, so it must be > 10, which it isn't. Passing down a valid (lo, hi) range at every level catches this.

D2.2

Take the middle element as root, recurse on the left half and right half. This gives height $\lceil \log_2 n \rceil$ (or $\lceil \log_2(n+1) \rceil - 1$), which is optimal since any binary tree holding n nodes needs height at least $\lceil \log_2(n+1) \rceil - 1$.

Inserting the same n sorted keys one at a time into an empty BST instead produces a completely skewed tree (each new key becomes the right child of the previous largest), giving height $n - 1$ — the worst possible, and search degrades to $O(n)$.

D3.1

```

void rangeQuery(TreeNode* node, int a, int b, vector<int>& out) {
    if (!node) return;
    if (node->val > a) rangeQuery(node->left, a, b, out);
    if (a <= node->val && node->val <= b) out.push_back(node->val);
    if (node->val < b) rangeQuery(node->right, a, b, out);
}

```

Time: $O(k + h)$, where h is the tree's height and k is the number of keys reported. The BST property lets the algorithm prune entire subtrees that provably lie outside $[a, b]$ (e.g., if $node->val \leq a$, the left subtree is skipped entirely), so it does not need to visit all n nodes — only those on the search paths to a and b , plus the k reported nodes.

D3.2

```

TreeNode* buildBST(std::queue<int>& preorder, int bound) {
    if (preorder.empty() || preorder.front() >= bound) return nullptr;
    int val = preorder.front(); preorder.pop();
    TreeNode* root = new TreeNode(val);
    root->left = buildBST(preorder, val); // must be < val
    root->right = buildBST(preorder, bound); // must be < bound (but > val, guaranteed by pre-
    order + BST property)
    return root;
}

```

Why $O(n)$: each call either consumes one element from the queue and creates a node, or returns immediately without consuming anything (at most one such 'wasted' call per real node, when a subtree is empty). So the total work across the whole recursion is proportional to the number of elements, $O(n)$ — not $O(n \log n)$, because the bound parameter lets each

element be placed correctly in a single pass without any repeated top-down comparisons against multiple ancestors (which is what would cost $O(\log n)$ extra per element in a naive repeated-insert approach).

Section E: Heaps & Priority Queues

E1.1

(a) [2,4,3,8,5,9,7]: valid min-heap. Check each parent i against children $2i+1$, $2i+2$: $2 \leq 4, 3$; $4 \leq 8, 5$; $3 \leq 9, 7$ — all hold.

(b) [2,5,3,4,8,9,7]: NOT valid — node at index 1 (value 5) has child at index 3 (value 4), and $5 > 4$ violates the min-heap property.

(c) [2,4,3,1,5,9,7]: NOT valid — node at index 1 (value 4) has child at index 3 (value 1), and $4 > 1$ violates the property.

E1.2

Array (0-indexed): [11, 4, 8, 2, 9, 1, 6, 3], $n=8$. Non-leaf indices: 3,2,1,0 (processed right-to-left).

$i=3$ (val 2, child $\text{idx}7=3$): $2 < 3$, no swap. Array unchanged: [11,4,8,2,9,1,6,3]

$i=2$ (val 8, children $\text{idx}5=1$, $\text{idx}6=6$): smallest child is 1; swap $8 \leftrightarrow 1$: [11,4,1,2,9,8,6,3]; 8 now at $\text{idx}5$, a leaf, stop.

$i=1$ (val 4, children $\text{idx}3=2$, $\text{idx}4=9$): smallest child is 2; swap $4 \leftrightarrow 2$: [11,2,1,4,9,8,6,3]; 4 now at $\text{idx}3$ (val 4, child $\text{idx}7=3$): swap $4 \leftrightarrow 3$: [11,2,1,3,9,8,6,4]; $\text{idx}7$ is a leaf, stop.

$i=0$ (val 11, children $\text{idx}1=2$, $\text{idx}2=1$): smallest child is 1 ($\text{idx}2$); swap $11 \leftrightarrow 1$: [1,2,11,3,9,8,6,4]; 11 now at $\text{idx}2$ (children $\text{idx}5=8$, $\text{idx}6=6$): smallest is 6 ($\text{idx}6$); swap $11 \leftrightarrow 6$: [1,2,6,3,9,8,11,4]; $\text{idx}6$ is a leaf, stop.

Final heap array: [1, 2, 6, 3, 9, 8, 11, 4].

E1.3

500 nodes: height = $\lfloor \log_2 500 \rfloor = 8$ (since $2^8=256 \leq 500 < 512=2^9$). Leaves occupy the last level, i.e. array indices from $\lfloor 500/2 \rfloor = 250$ through 499 (0-indexed) are the non-leaf/leaf boundary — more precisely, leaves are all indices i with $2i+1 \geq 500$, i.e. $i \geq 250$, so leaves occupy indices 250 through 499.

E2.1

Best: maintain a min-heap of size k while scanning once, $O(n \log k)$ time, $O(k)$ space — push each element, and whenever the heap exceeds size k , pop the minimum; the heap's root at the end is the k -th largest.

Sorting costs $O(n \log n)$ — worse when k is small. QuickSelect averages $O(n)$ but has $O(n^2)$ worst case (e.g. adversarial/already-sorted input with poor pivot choice), and it needs the whole array in memory / requires multiple passes over the same buffer, so it doesn't work on a single-pass stream, whereas the heap approach naturally does (only k elements held at any time).

E2.2

Push the head of each of the k lists into a min-heap keyed by value (size $\leq k$). Repeatedly pop the minimum, append it to the output, and if the popped node has a next node, push that into the heap. Continue until the heap is empty.

Each of the n total elements is pushed and popped from the heap exactly once, each operation costing $O(\log k)$, giving total time $O(n \log k)$.

E2.3

A monotonic deque (double-ended queue storing indices, kept in decreasing order of value) achieves $O(n)$ total time: for each new element, pop from the back while it is smaller than the incoming element (those elements can never be the max of any future window containing the new one), then push the new index; pop from the front if it has fallen outside the current window. Each index is pushed and popped at most once, giving $O(n)$ total — better than a heap's $O(n \log k)$, because a heap doesn't efficiently support removing an arbitrary element that has aged out of the window without extra bookkeeping (e.g. lazy deletion).

E3.1

Maintain a max-heap 'lo' holding the smaller half of numbers seen so far, and a min-heap 'hi' holding the larger half, with the invariant that sizes differ by at most 1 (and, if unequal, lo holds one more element than hi).

$\text{addNum}(x)$: push x onto the appropriate heap based on comparison with lo's max (or default to lo if empty), then rebalance by moving the top of one heap to the other if the size invariant is violated. All $O(\log n)$.

findMedian(): if sizes are equal, average of lo's max and hi's min; otherwise, lo's max (since lo has the extra element). $O(1)$.

E3.2

Store, alongside the heap array, a hash map from key to its current index in the array. Whenever an element's priority changes, look up its array index in $O(1)$ via the map, then bubble it up or down as needed ($O(\log n)$); the map must also be updated for every element that gets swapped during that bubbling, but each swap only updates two map entries, so the overall cost stays $O(\log n)$. This is exactly the technique used to support a decrease-key operation, as needed e.g. in Dijkstra's algorithm.

Section F: Mixed Problems & Proofs

F1.1

- (a) A (max- or min-) heap / priority queue — gives $O(\log n)$ insert and $O(\log n)$ extraction of the most urgent task.
- (b) Two stacks (an 'undo' stack and a 'redo' stack) — each action pushed onto undo; undoing pops from undo and pushes onto redo, and vice versa, all $O(1)$.
- (c) A hash set (`std::unordered_set`) — gives expected $O(1)$ membership checks, which is faster on average than a BST's $O(\log n)$.
- (d) A min-heap of size 10 — keep the 10 largest seen so far; compare each new value to the heap's minimum and replace if larger, giving $O(\log 10) = O(1)$ per element and $O(n)$ total for a stream of size n .

F1.2

Not sound as stated. `push_back` is $O(1)$ amortized, not $O(1)$ worst-case — an individual call can cost $O(n)$ when the vector must reallocate and copy every existing element. What IS true is that the total cost of n calls to `push_back` is $O(n)$ (by the amortized analysis), so the loop as a whole runs in $O(n)$ time — but that conclusion follows from amortized analysis over the whole loop, not from each call individually being $O(1)$. The distinction matters if, e.g., you were trying to bound the worst-case latency of a single iteration (which can spike to $O(n)$), not just the total running time.

F2.1

Process elements in pairs: for each pair, 1 comparison determines a local min and max candidate ($\lfloor n/2 \rfloor$ comparisons total for n even, or one element left over). Then compare the running overall minimum against each pair's local min (1 comparison per pair) and the running overall maximum against each pair's local max (1 comparison per pair): 2 more comparisons per pair. Total $\approx \lfloor n/2 \rfloor + 2 \cdot \lfloor n/2 \rfloor = 3\lfloor n/2 \rfloor$ comparisons in a naive count, but a tighter accounting (updating a running min/max together per pair rather than restarting) gives exactly $\lceil 3n/2 \rceil - 2$, which matches the known lower bound derivable via an adversary argument: each element must be compared at least once to become a min-candidate and at least once to become a max-candidate, except the very first pair shares one comparison between establishing both.

F3.1

Modify merge sort: while merging two already-sorted halves, whenever an element from the right half is placed into the output before an element from the left half, it means that right-half element is smaller than ALL remaining elements in the left half (since the left half is sorted) — so add (number of remaining left-half elements) to the inversion count at that step. Recursing this way, the total additional work is the same $O(n \log n)$ as merge sort itself, since inversion-counting only adds $O(1)$ extra work per merge step.

F3.2

For each i from 0 to $n-1$, the inner loop rebuilds a BST from scratch with $i+1$ elements, doing $i+1$ inserts each costing $O(\text{height})$. If insertions happen to keep the tree balanced, each insert is $O(\log i)$, so building costs $O(i \log i)$ at iteration i , giving total $\sum O(i \log i)$ for $i=1..n \approx O(n^2 \log n)$ overall (dominated by the outer loop's $O(n)$ iterations, each doing up to $O(n \log n)$ work). In the worst case (insertion order causes a skewed tree, e.g. already-sorted input), each insert costs $O(i)$, so building costs $O(i^2)$ at iteration i , and the total becomes $\sum O(i^2)$ for $i=1..n = O(n^3)$. Either way, the naive full-rebuild-every-iteration approach is far more expensive than maintaining one incrementally-updated balanced BST across iterations would be.